



Mahavir Education Trust's

# Shah & Anchor Kutchhi Engineering College

Chembur, Mumbai-400088

(An Autonomous Institute Affiliated to University of Mumbai)



## Department of Electronics & Communication (Advanced Communication Technology)

*Signals of Progress: Unleash the Power of Advanced Communications*

presentation  
on

### **Dimensionality Reduction: T-SNE**

(Subject: Advanced Machine Learning (ACCOR1PC201), VI Semester)

by

**Manav Nimish Kothari**  
**23UF18402AC019**

# Introduction to Dimensionality Reduction

## What is it?

Dimensionality reduction in machine learning is the process of reducing the number of input variables (features) in a dataset while preserving important information.



- Imagine you have a **Big LEGO house** with **100 floors** (So now that's your data with many "dimensions").
- It's **too big to carry and show** to your friends
- So you make a **small LEGO model** with only **2 floors** but it still shows the **important shape** of the house



That's what **dimensionality reduction** does → it makes **big, complicated data** smaller and simpler so we can **see and understand it**.

# T-SNE

## What is it?

t-SNE, which stands for t-Distributed Stochastic Neighbor Embedding, is a non-linear dimensionality reduction algorithm widely used in machine learning for visualizing high-dimensional data.



Imagine you have a big **box of LEGO bricks**.

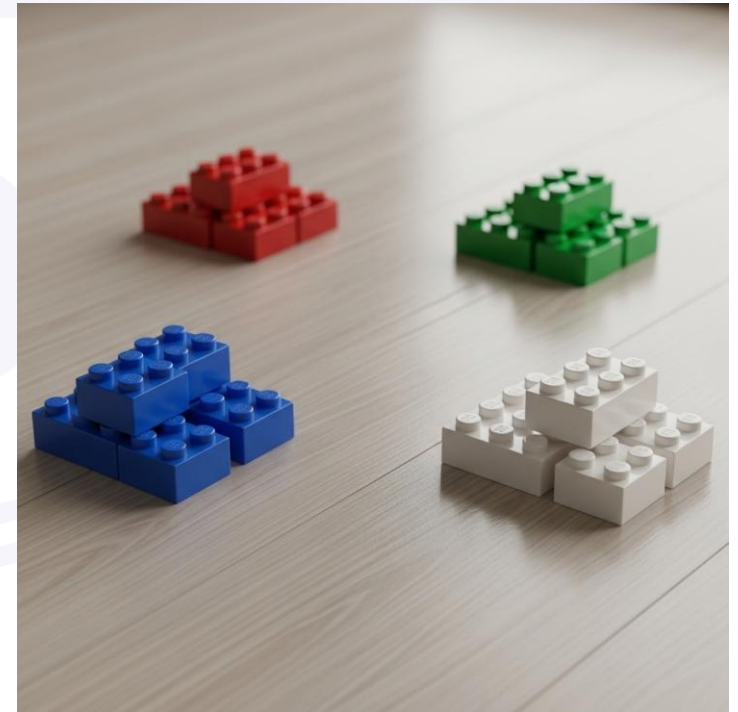
Some are **red**, some are **blue**, some are **green**.

They also come in **different shapes** long, short, square, thin.

t-SNE is like a **magic sorter**.

It takes your big messy box and puts the LEGO bricks on the floor in such a way that

Bricks that look **similar** (say, small red ones) are placed **close together**.



t-SNE = **Magic map maker** that puts **similar things together** and **different things apart**, so our eyes can understand complicated stuff easily.

# How t-SNE Works

- **Compute similarities in high dimensions**

Closer points get higher similarity.

- **Make similarities mutual**

Both points should “agree” they are close.

- **Map to low-dimensional space (2D/3D)**

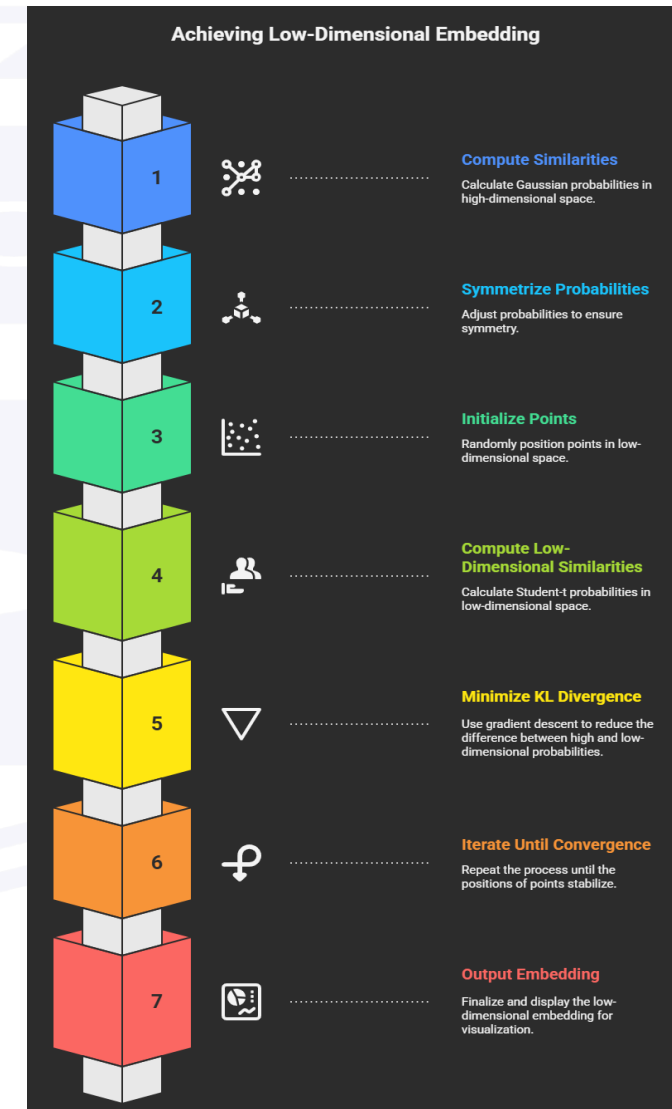
Start with random positions.

- **Adjust positions**

Similar points pulled closer, dissimilar points pushed apart.

- **Final result**

Clear 2D/3D clusters that reflect original structure.



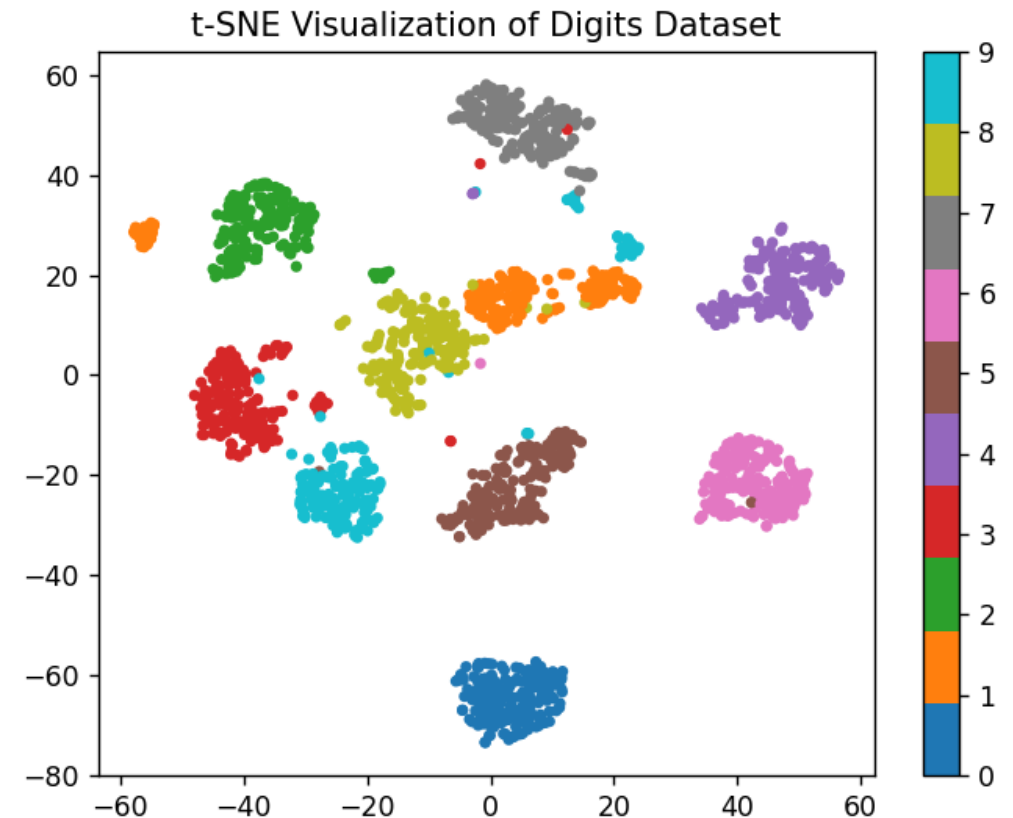


# Example- t SNE (MNIST)

Modified National Institute of Standards and Technology dataset

```
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.data
y = digits.target
tsne = TSNE(n_components=2, perplexity=30, learning_rate=200, n_iter=1000, random_state=42)
X_embedded = tsne.fit_transform(X)

plt.scatter(X_embedded[:,0], X_embedded[:,1], c=y, cmap="tab10", s=10)
plt.colorbar()
plt.title("t-SNE Visualization of Digits Dataset")
plt.show()
```



# Mathematical Insight of t-SNE

Cost Function (to minimize):

$$KL(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- **High-dimensional similarities ( $p_{ij}$ ):** Computed using a Gaussian distribution of distances between points.
- **Low-dimensional similarities ( $q_{ij}$ ):** Computed using a Student-t distribution (heavy tails avoid crowding).

## Advantages of t-SNE

- Great for visualization
- Captures non-linear patterns
- Preserves local relationships
- Useful for clustering insights

## Limitations of t-SNE

- Computationally expensive
- Cannot be used for prediction (only visualization)
- Results can vary depending on parameters
- Not ideal for very large datasets

# Where can we use t-SNE ?

## Data Visualization:

t-SNE is mainly used to **visualize high-dimensional data** in 2D or 3D.

Example: Visualizing images, text embeddings, or sensor data.

## Cluster Exploration:

Helps to **see natural groups or clusters** in data.

Example: Grouping handwritten digits, gene expression profiles, or customer segments.

## Understanding Embeddings:

Used to **inspect feature embeddings** from deep learning models.

Example: Word embeddings (Word2Vec, BERT) or image embeddings from CNNs.

## Anomaly Detection:

t-SNE can show **outliers** that are far from clusters.

Example: Detecting unusual user behavior or rare events in data.

## Preprocessing Check:

Helps check **if preprocessing or feature engineering worked well**.

Example: After PCA or normalization, t-SNE can show if clusters are forming.



# References

- **DataCamp – t-SNE Tutorial**  
<https://www.datacamp.com/tutorial/t-sne-python>
- **GeeksforGeeks – t-SNE in Machine Learning**  
<https://www.geeksforgeeks.org/tsne-in-python/>
- **KDnuggets – Introduction to t-SNE**  
<https://www.kdnuggets.com/2020/06/introduction-t-sne.html>